

ARRAYS

STORING MARKS OF 5 STUDENTS

We are given 5 numbers in the input, which represent the marks of 5 students. We just need to store them in our program.

```
int main()
{
    int m1, m2, m3, m4, m5;
    cin >> m1 >> m2 >> m3 >> m4 >> m5;
}
```

STORING MARKS OF 10 STUDENTS

We are given 10 numbers in the input, which represent the marks of 10 students. We just need to store them in our program.

```
int main()
{
    int m1, m2, m3, m4, m5, m6, m7, m8, m9, m10;
    cin >> m1 >> m2 >> m3 >> m4 >> m5 >> m6 >> m7 >> m8 >> m9 >> m10;
}
```

STORING MARKS OF 100 STUDENTS

We are given 100 numbers in the input, which represent the marks of 100 students. We just need to store them in our program.

```
// impossible and very hard to manage
int main()
{
    int m1, m2, m3, ... m100;
    cin >> m1 >> m2 >> m3 >> ... .. >> m100;
}
```



STORING MARKS OF 100 STUDENTS

We are given 100 numbers in the input, which represent the marks of 100 students. We just need to store them in our program.

```
int main()  
{
```

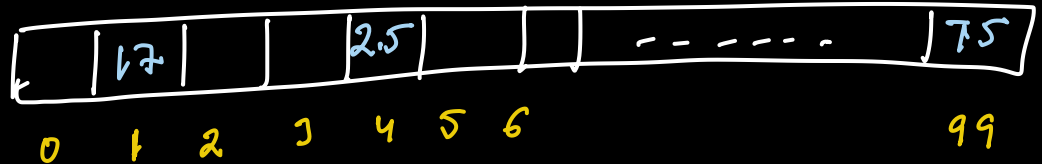
```
    int marks[100];
```

```
    marks[1] = 17;
```

```
    marks[4] = 25;
```

```
    marks[99] = 75;
```

marks



creating an array

```
[ cin >> marks[0] ;  
  cin >> marks[1] ;  
  cin >> marks[2] ;  
  :  
  :  
  :  
  cin >> marks[99] ;  
]
```

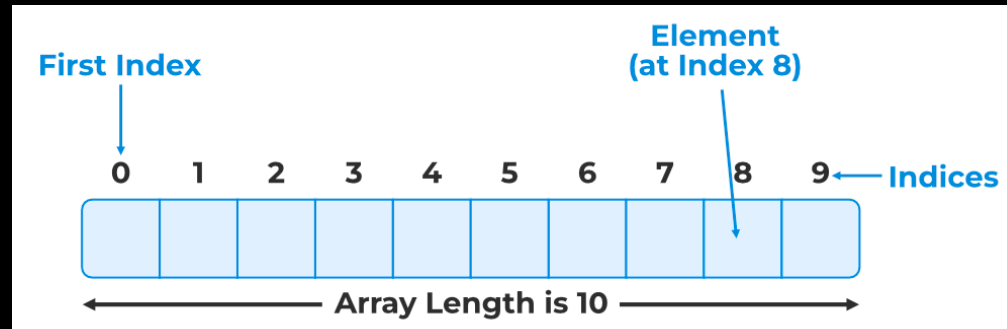
⇒

```
for (i = 0; i <= 99; i++)  
{  
    cin >> marks[i];  
}
```

}

ARRAYS

- A **collection** of **similar kind of data** stored together at **continuous memory locations**.
- It helps us **create multiple variables** of the **same data type** using a **single name**.
- Each element can be accessed using the **array name** along with its **unique index**.
- Indexing starts from **0**, and the last element is located at index **$N - 1$** , where **N** is the total number of elements in the array.



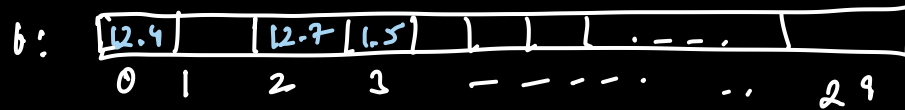
CREATING AN ARRAY

- **DataType** **ArrayName**[**NUMBER_OF_ELEMENTS**];

◦ **int** a[20];



◦ **float** b[30];



◦ **long long** c[500];



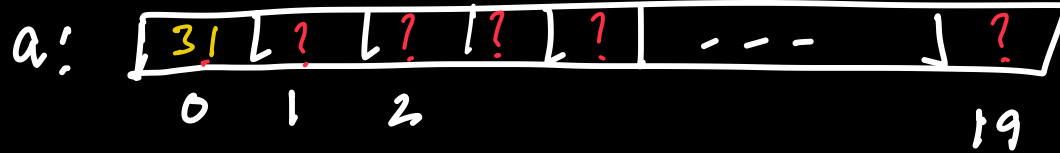
- Once you create an array, all elements can have any random value (often referred as **garbage value**)

ACCESSING ELEMENTS

0 to N-1

18

- `int a[20];`



- `a[0] = 31;`

- `a[1]++;` ✓

- `cout << a[2] << endl;` // Garbage Value

`a[1]` ✓

`a[18]` ✓

`a[20]` ✗

`a[-1]` ✗

- Array indices are from 0 to (N-1), if you try to access `a[i]` where `i` is not a valid index (i.e. `i < 0` or `i >= N`), it will lead to undefined behaviour, could be a runtime error, segmentation fault or accessing any garbage value.

INPUT / OUTPUT

5

7 8 1 2 4

7 8 1 2 4

```
int main()
{
    int n;
    cin >> n;

    int A[n];
    for(int i = 0; i < n; i++)
    {
        cin >> A[i];
    }

    for(int i = 0; i < n; i++)
    {
        cout << A[i] << " ";
    }
}
```

PRINT ARRAY IN REVERSE

5

7 8 1 2 4

4 2 1 8 7

FIND MAX OF THE ARRAY

6
1 4 8 7 9 3

6
1 4 8 7 9 3

9

9

FIND MIN OF THE ARRAY

6
7 6 8 9 1 3

1

SEARCH IN AN ARRAY

6 10

7 6 8 9 1 3

6 8

7 6 8 9 1 3

NOT FOUND

FOUND

COUNT OCCURENCES

6 7
7 6 7 9 7 3

6 8
7 6 8 9 1 3

3

1

CHECK IF THE GIVEN ARRAY IS SORTED

6

1 2 2 8 11 31

6

7 6 7 9 7 3

YES

NO

Sort 01

6

110110

6

100110

001111

000111

SWAP 2 NUMBERS

$A = 2$
 $B = 5$

$A = 5$
 $B = 2$

SWAP ALTERNATE

5

7 8 1 2 4

6

7 9 8 1 2 4

8 7 2 1 4

9 7 1 8 4 2

REVERSE IN THE SAME ARRAY

5

7 8 1 2 4

6

7 9 8 1 2 4

4 2 1 8 7

4 2 1 8 9 7

MISSING NUMBER

Every number in the array is present twice, except for one special element that occurs only once. Find the special element.

7
2 1 9 1 2 3 9

7
1 2 1 2 3 4 3

3

4

INTERSECTION OF TWO ARRAYS

Print the elements present in both arrays.

6

2 6 8 5 4 3

4

2 3 4 7

4

2 6 1 2

5

1 2 3 4 2

2 4 3

2 2

PAIR SUM

Count how many pairs in the array satisfies: $A[i] + A[j] = \text{target}$

9

1 3 6 2 5 4 3 2 4

7

7

TRIPLET SUM

Count how many triplets in the array satisfies: $A[i] + A[j] + A[k] = \text{target}$

7

1 2 3 4 5 6 7

12

5

QUADRAPLET SUM

Count how many quadruplets in the array satisfies: $A[i] + A[j] + A[k] + A[l] = \text{target}$

